Algorithm 1: $\texttt{TrainingStorage}_g \leftarrow \texttt{SharedResponseRollout}(\theta_k, \bar{\theta}, G_k)$

Input: one trainable shared response π^{θ_k}; one read-only target $\bar{\pi} = \pi^{\bar{\theta}}$ bound to its content hash; the sequential-RB partition, queue, admission, logical-RNG, and completion contracts
Output: ordinary sequential-RB training storage containing selected-seat rows only
Allocate exactly the sequential-RB owners admitted by immutable schemas and realized work; keep $\bar{\theta}$ read-only for the entire run; **for** $i = 0, \ldots, G_k - 1$ **do**
  Draw $I_i \sim \mathsf{Unif}(\mathcal{P})$ from $K(k, i, \mathsf{probeSeat})$ before private-state and event draws; Attach I_i to the live root owner; it remains phase-local rollout control;
end
Run the family-scheduled sequential queue from `family_scheduled_rollout.tex`, with these role overrides:
(i) when a player occurrence s from root i has $o(\ell(s)) = I_i$, forward π^{θ_k} and append its natural input, owner, sampled event, old log probability, and old value;
(ii) otherwise forward $\bar{\pi}$ and use its sampled event and reach update only to materialize the successor; append no decision/model row;
(iii) after a selected-seat action, test the game-owned public cutoff predicate. If it holds, write the synthetic owner PROBE_FOLD on the occurrence; never attempt to derive that owner from the underlying game-state phase.
At each stopped leaf, capture in one last-consumer transaction the occurrence owner, I_i, stopped reach, logical completion key, and immutable completion inputs. At a synthetic cutoff write only the selected payoff $F_{I_i}(\ell(s))$ from Eq. (probe-public-cutoff); at an ordinary leaf use ordinary sequential-RB settlement. Release the mutable state, root profile, selected-seat control, and opponent transient immediately after their last captured consumer;
Seal one `SequentialDecisionCSR` and one leaf per root. Build targets by `BuildSequentialRBTargets` and finalize by the ordinary sequential-RB schema. Assert every retained decision owner equals its root's selected seat, no opponent row exists, every synthetic cutoff retains its explicit owner and selected payoff, and no selected-seat/root-profile control crosses into `TrainingStorage`; **return** $\mathit{TrainingStorage}_g$;

Algorithm 2: $\widehat{\theta}_K \leftarrow \texttt{TrainSharedApproximateResponse}(\theta_0, \bar{\theta})$

Input: hash-bound target checkpoint $\bar{\theta}$; configured initialization θ_0; fixed iteration count K and optimizer steps per iteration
Output: finite-budget shared approximate response $\widehat{\theta}_K$ and work ledger
Verify the target content hash and identical game tuple (P, S, R) before model or rollout allocation; initialize the shared response randomly or from $\bar{\theta}$ exactly as configured; **for** $k = 0, \ldots, K - 1$ **do**
  Build selected-only stores with $\texttt{SharedResponseRollout}(\theta_k, \bar{\theta}, G_k)$ on every rank; Apply `DistributedFixedRolloutUpdate` with mode seqRB and Eq. (shared-probe-empirical-loss); update only θ_k; Add exact logical trajectories, completion samples, minibatches, and optimizer steps to the checkpointed work ledger;
end
Store $\widehat{\theta}_K = \theta_K$, the `shared_seat_approximate_response_v1` semantics identifier, target hash, initialization rule, scientific configuration, RNG state, optimizer state, and work ledger; `/* Budget exhaustion is the only stopping rule. Do not emit a convergence, best-response, exploitability, or NashConv certificate. */`
return $\widehat{\theta}_K$ *and ledger*;

Algorithm 3: $\texttt{Report} \leftarrow \texttt{EvaluateSharedApproximateResponse}(\widehat{\theta}_K, \bar{\theta}, H)$

Input: fixed probe and hash-matched target; $H > 1$ divisible by P^2
Output: paired shared-response lower-bound report
Assign every consecutive P^2 roots to all (selected seat, button) pairs exactly once; **foreach** *bounded evaluation root batch* **do**
  Run the selected-seat mixed profile and the all-target baseline with the same root identities and logical RNG namespaces; matching logical addresses reuse keys, while genuinely divergent paths retain their own keyed draws; Record paired selected-seat differences D_j and release both rollouts;
end
Compute each $\widehat{g}_p$ and $\widehat{\mathrm{NashConv}}_{\mathrm{probe}} = \sum_p \widehat{g}_p$ exactly as Eq. (approximate-nashconv-estimator). Assert every seat count is equal and positive and the sum equals P times the overall balanced mean; Compute the paired Student-t interval over $(D_j)_j$ and label it evaluation sampling uncertainty only; Report per-seat gains, approximate NashConv lower bound, hashes, seed, paired evaluation work, training logical-work ledger, and wall time; **return** Report;